



本科生实验报告

实验课程: 操作系统实验

实验名称: 实验 4

专业名称: 计算机科学与技术

学生姓名: 陈安康

学生学号: 22320021

实验地点: 实验中心 D402

实验成绩: _____

报告时间: 2024/4/17

1. 实验要求

学习混合编程，完成操作系统中断。

2. 实验过程

任务 1:

在学习了混合编程的相关知识之后，开始复现任务一，由要求可以知道，这是一个汇编语言首先调用 C/C++实现的函数，接着由 C/C++程序调用该汇编代码的一个事例。当汇编语言调用 C/C++代码时，需要首先在汇编代码中通过“extern <函数名>”的形式来声明 C/C++函数，与此同时为了避免声明的标号被改变，需要将 C++代码中的函数声明前面加上“extern “C””，来告诉编译器不进行名字修饰。如果当 C/C++代码调用汇编函数时，需要在汇编代码中将函数声明为 global，确保在连接时可以找到函数的实现，接着需要在 C/C++代码中将被调用的代码声明为来自外部，若为 C++代码则要声明为“extern “C””即按 C 风格编译，若有返回值则还需要考虑入栈出栈取参数和放参数等情况，因为需要实现的这个例子中没有函数有参数，所以这里不再赘述。

具体例子实现中，首先两个被汇编代码调用的函数正常实现，其中一个 C++代码要附上“extern C”。然后在汇编代码中声明如下：

```
global function_from_asm
```

```
extern function_from_C
```

```
extern function_from_CPP
```

其中第一个为以后这个汇编函数被 main 的 C/C++代码调用做准备，使得可以找得到该汇编函数，后面两个则是声明其所调用的函数来自 C/C++。

函数本身实现十分简单，这里不再赘述，实现完成之后只要一一进行编译即可，可是这样太过于麻烦，因此需要自己编写一个 Makefile 文件。首先需要明确这个例子的工作空间如下：



```
Done.  
user@user-virtual-machine:~/lab4/example$ tree  
.  
├── build  
│   ├── asm_func.o  
│   ├── c_func.o  
│   ├── cpp_func.o  
│   └── main.o  
├── main.out  
├── Makefile  
└── src  
    ├── asm_func.asm  
    ├── c_func.c  
    ├── cpp_func.cpp  
    └── main.cpp  
  
2 directories, 10 files
```

Build 文件夹放置所有中间文件，src 文件夹放置源代码，Makefile 和最终生成的可执行文件与 build 和 src 文件夹放在同一级目录下，因此 makefile 文件要做的是将所有的 src 文件中的源代码编译生成对应的中间文件放在 build 文件夹中并且再次编译生成最终的 main.out 文件。

自己编写的 Makefile 内容如下：

```
user@user-virtual-machine: ~/lab4/example

BUILD_DIR = build
SRC_DIR = src

main.out : $(BUILD_DIR)/main.o $(BUILD_DIR)/c_func.o $(BUILD_DIR)/cpp_func.o $(BUILD_DIR)/asm_func.o
    g++ -o main.out $(BUILD_DIR)/main.o $(BUILD_DIR)/c_func.o $(BUILD_DIR)/cpp_func.o $(BUILD_DIR)/asm_func.o -m32

$(BUILD_DIR)/c_func.o : $(SRC_DIR)/c_func.c
    gcc -o $(BUILD_DIR)/c_func.o -m32 -c $(SRC_DIR)/c_func.c

$(BUILD_DIR)/cpp_func.o : $(SRC_DIR)/cpp_func.cpp
    g++ -o $(BUILD_DIR)/cpp_func.o -m32 -c $(SRC_DIR)/cpp_func.cpp

$(BUILD_DIR)/main.o : $(SRC_DIR)/main.cpp
    g++ -o $(BUILD_DIR)/main.o -m32 -c $(SRC_DIR)/main.cpp

$(BUILD_DIR)/asm_func.o : $(SRC_DIR)/asm_func.asm
    nasm -o $(BUILD_DIR)/asm_func.o -f elf32 $(SRC_DIR)/asm_func.asm

clean :
    rm $(BUILD_DIR)/*.o *.out
"Makefile" 21L, 717C 1,1 全部
```

main.out 执行结果如下：

```
user@user-virtual-machine: ~/lab4/example

user@user-virtual-machine:~$ cd lab4
user@user-virtual-machine:~/lab4$ cd example/
user@user-virtual-machine:~/lab4/example$ cd build
user@user-virtual-machine:~/lab4/example/build$ ls
asm_func.o  c_func.o  cpp_func.o  main.o
user@user-virtual-machine:~/lab4/example/build$ cd ..
user@user-virtual-machine:~/lab4/example$ ls
build  main.out  Makefile  src
user@user-virtual-machine:~/lab4/example$ make
make: "main.out"已是最新。
user@user-virtual-machine:~/lab4/example$ ./main.out
Call function from assembly.
This is a function from C.
This is a function from C++.
Done.
user@user-virtual-machine:~/lab4/example$
```

代码位于 example 文件夹中。

任务 2:

经过学习可以知道，内核的加载是在保护地址模式下

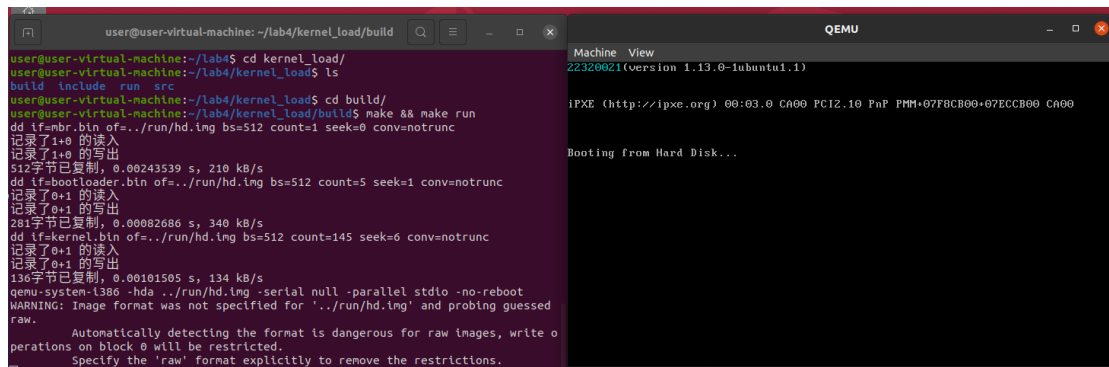
进行的,所以需要首先通过 MBR 将 Bootloader 加载进内存,再在 Bootloader 中完成从 16 位实地址模式到 32 位保护地址模式的跳转,跳转完成后再将内核程序加载到内存中。由于前面的两个步骤已经在 lab2 和 lab3 完成,这里只需要关注加载内核进入内存并通过内核完成我们想要的操作即可。

因为规定了内核起始地址为 0x20000,所以我们需要在 Bootloader 中远跳转到该地址,然后执行我们的内核程序,(内核程序的加载地址是在链接时指定的),为了保证内核二进制文件的起始地址是内核进入地址,由于链接时的顺序与命令中给出文件的顺序相同,因此我们只需要将入口代码放在第一个位置上就可以了。

从入口代码中指定要跳转到的标号(这个标号就是用 C/C++实现的函数名),这样就可以跳转到 C/C++代码中来进行内核程序的实现。

由于内核的任务是打印一段指定的文字,而现在没有办法用 C 的标准库来实现,所以用 C 来实现打印有些困难,而由于之前的积累,汇编实现字符打印十分简单,所以这里采用 C 中调用一个汇编函数来完成任务。接下来就是注意一下混合编程中所需要的声明即可。

实现自己的汇编函数打印自己的学号,实现结果如下:



```
user@user-virtual-machine: ~/lab4/kernel_load/build
user@user-virtual-machine:~/lab4$ cd kernel_load/
user@user-virtual-machine:~/lab4/kernel_load$ ls
build include run src
user@user-virtual-machine:~/lab4/kernel_load$ cd build/
user@user-virtual-machine:~/lab4/kernel_load/build$ make && make run
dd if=nbr.bin of=../run/hd.img bs=512 count=1 seek=0 conv=notrunc
记录了1+0 的读入
记录了1+0 的写出
512字节已复制, 0.00243539 s, 210 kB/s
dd if=bootloader.bin of=../run/hd.img bs=512 count=5 seek=1 conv=notrunc
记录了0+1 的读入
记录了0+1 的写出
261字节已复制, 0.00082686 s, 340 kB/s
dd if=kernel.bin of=../run/hd.img bs=512 count=145 seek=6 conv=notrunc
记录了0+1 的读入
记录了0+1 的写出
136字节已复制, 0.00101505 s, 134 kB/s
qemu-system-i386 -hda ../run/hd.img -serial null -parallel stdio -no-reboot
WARNING: Image format was not specified for '../run/hd.img' and probing guessed
raw.
Automatically detecting the format is dangerous for raw images, write o
perations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.

Machine View
22320021(version 1.13.0-1ubuntu1.1)

iPXE (http://ipxe.org) 00:03:0 CA00 PC12.10 FnP PMM+07F8CB00+07ECCB00 CA00

Booting from Hard Disk...
```

代码位于 kernel_load 文件夹中。

任务 3:

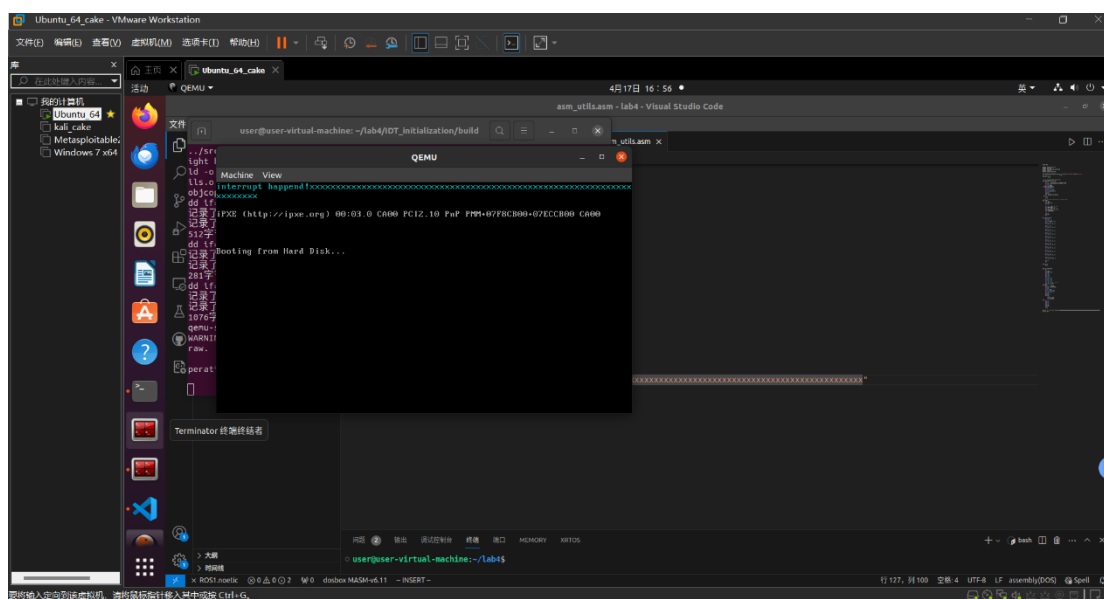
初始化 IDT 需要完成三个步骤：确定 IDT 的地址、定义中断默认处理函数、初始化中断描述符。为了方便实现与管理，我们定义一个类来管理中断。IDT 的地址我们规定在 0x8880 处，并且需要将它的大小和初始地址放在寄存器 IDTR 中。这个过程在 C 中难以实现，所以要调用汇编代码进行实现。其中涉及含参数的汇编函数的调用，需要先初始化堆栈栈底指针，然后从堆栈中取出想要的参数（按照入栈顺序推算即可，当前初始化的栈底指针是指向的上一个栈底指针的位置，再上面是返回地址，再往上就是参数了，由于栈是从高往低扩展，所以取参数需要做加法而不是减法）汇编实现过程与 GDTR 的初始化基本一致，不再赘述。

接下来就是定义默认中断处理函数。在这个地方我深刻体会到了汇编的优势，因为刚开始我想着用纯 c 代码完成中断处理函数，但是由于自己能力原因，我不知道怎么用纯 C 语言在不借助任何第三方库的情况下实现中断的开关。所以我准备退而求其次，在 C 语言中调用自己编写好的汇编函数，

只要在汇编函数中实现开关中断不就好了？可是在实际实现中我发现它会报一些莫名其妙的错误（大概率是由于套了一层 C 语言的“外皮”导致中断嵌套了，因为每次都是在 C 语言中途发生了一个未知中断，因此推断是嵌套了导致错误，当然，这是采用 `iret` 命令返回时的情况，如果最后的汇编代码是死循环，那么就会看起来完全没有问题，我感觉这样违背该实验的初心，所以没有采纳），因此我只好纯汇编语言来实现这个自定义的代码，代码功能十分简单，仅仅打印自己的字符串，并且仅用这个函数初始化中断向量表中第一个中断处理函数即除零中断处理函数。（这里有一个额外发现，或许可以解释为什么示例中给出的中断处理函数也是做死循环，因为中断处理函数若什么都没做只是打印字符返回后，还会返回到除零中断处，而这个错误还是存在的，所以它会一直不断地执行这个中断处理函数，所以我想这就是为什么示例代码中到这个位置选择不是开中断返回而是做死循环）

最后是初始化 IDT，只需要将中断描述符需要的信息送入中断描述符中即可。

结果如下：



代码位于 IDT_initialization 文件夹中。

任务 4:

实现时钟中断需要用到 8259A 芯片，通过教程的学习我知道了 8259A 芯片是用来处理中断请求的可编程中断处理器，通过初始化命令字来对它进行初始化，并且要在其处理中断返回前向 8259A 芯片手动发送 EOI 消息，其中的优先级、中断屏蔽字和 EOI 消息都可以通过 OCW 字进行设置或者发送。

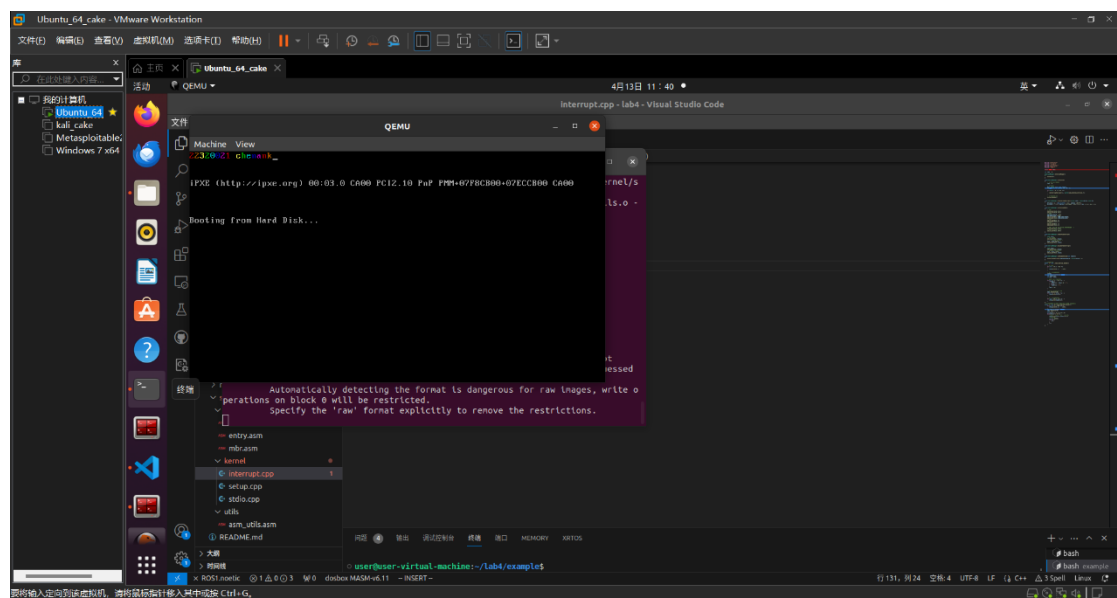
具体实现上，在 lab3 创建的中断管理器中初始化 8259A 芯片，实现方法是向规定的外设端口发送初始化命令字来完成（这里已经在 8259A 中设置好了中断向量号）。

接下来是编写中断处理函数，这里我想法是输出一段包括我学号和 netID 的字符，为了实现走马灯的效果，因为时钟中断会不停地触发，意味着我的代码会周期性的执行，所以我只要完成一个字符一个字符地输出的效果即可，在之前的实验中通过设置制造延迟 nop 已经实现了，这里在 C 语言下

编程，采用的是空循环来制造延时，同时实现了字符颜色按照彩虹七色来显示的效果，即将彩虹每个颜色的对应颜色的显存数据放在一个数组中。显示使用的是示例中定义好的显示类。在这里是采用的汇编调用 C 语言的形式，因为 C 语言缺少中断实现的语法。

设置一下时钟中断的中断描述符，这个方法和初始化 IDT 的过程中的方法是一样的。实现一下时钟中断的开启与关闭函数，通过读取和写入 ocw1 来实现。

运行结果如下图：



这个效果在黑屏上比较明显，linux 原始 shell 屏幕可能会导致一些字符显示不清。

代码位于 timer_interrupt 文件夹中。

3. 关键代码

任务 1:

任务 2:

自己实现的汇编代码打印自己的学号如下：

```
[bits 32]

global asm_hello_world

asm_hello_world:
    push eax
    push ebx
    push esi
    push ecx
    xor eax, eax
    xor ebx, ebx
    xor esi, esi
    xor ecx, ecx
    mov ecx, my_id_end - my_id
    mov ah, 0x3
    mov esi, my_id
my_loop:
    mov al, [esi]
    mov word[gs:ebx], ax
    inc esi
    add ebx, 2
    loop my_loop
    pop ecx
    pop esi
    pop ebx
    pop eax
    ret
my_id db '22320021'
my_id_end:
```

该代码会打印 my_id 中的字符串到屏幕上，意味着可以修改 my_id 指向任意字符串，其都可以进行打印。

任务 3:

任务 4:

走马灯代码如下：

```

23 //自定义的时钟中断函数，实现走马灯显示学号和姓名的效果
24 extern "C" void my_c_time_interrupt_handler() {
25     for(int i = 0; i < 80; i++) {
26         stdio.print(0, i, ' ', 0x07);
27     }
28     char str[] = "22320021 chenank";
29     stdio.moveCursor(0);
30     int i = 0;
31     int color[7] = {4,6,14,2,3,1,5};
32     while(str[i] != '\0') {
33         //color = (color * 317 + ) & 0x0f;
34         stdio.print(str[i], color[i % 7]);
35         //延迟用的
36         int j = 1000000;
37         while(j) {
38             j--;
39         }
40         ++i;
41     }
42 }

```

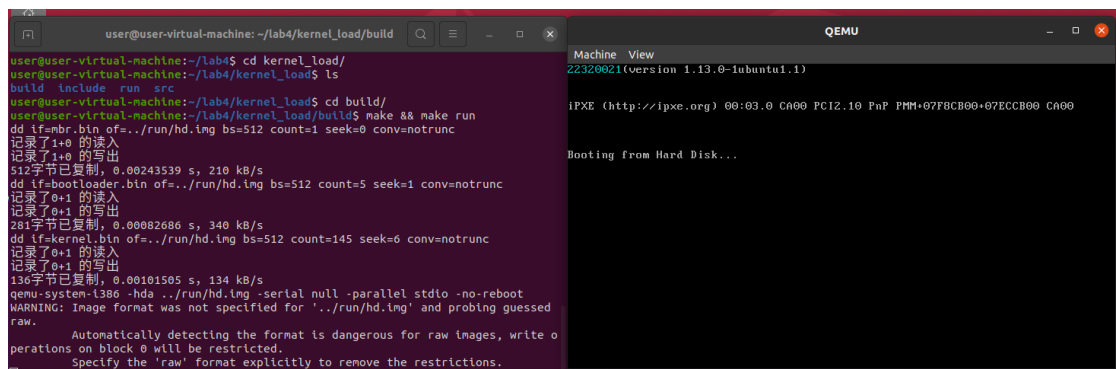
4. 实验结果

```

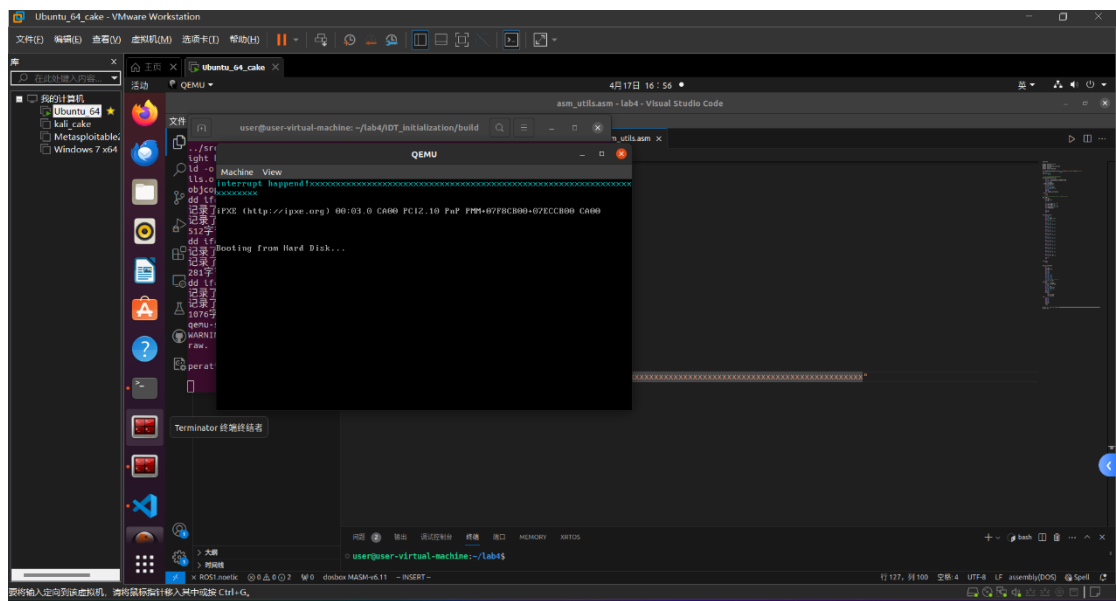
user@user-virtual-machine: ~/lab4/example
user@user-virtual-machine:~$ cd lab4
user@user-virtual-machine:~/lab4$ cd example/
user@user-virtual-machine:~/lab4/example$ cd build
user@user-virtual-machine:~/lab4/example/build$ ls
asm_func.o  c_func.o  cpp_func.o  main.o
user@user-virtual-machine:~/lab4/example/build$ cd ..
user@user-virtual-machine:~/lab4/example$ ls
build  main.out  Makefile  src
user@user-virtual-machine:~/lab4/example$ make
make: "main.out"已是最新。
user@user-virtual-machine:~/lab4/example$ ./main.out
Call function from assembly.
This is a function from C.
This is a function from C++.
Done.
user@user-virtual-machine:~/lab4/example$

```

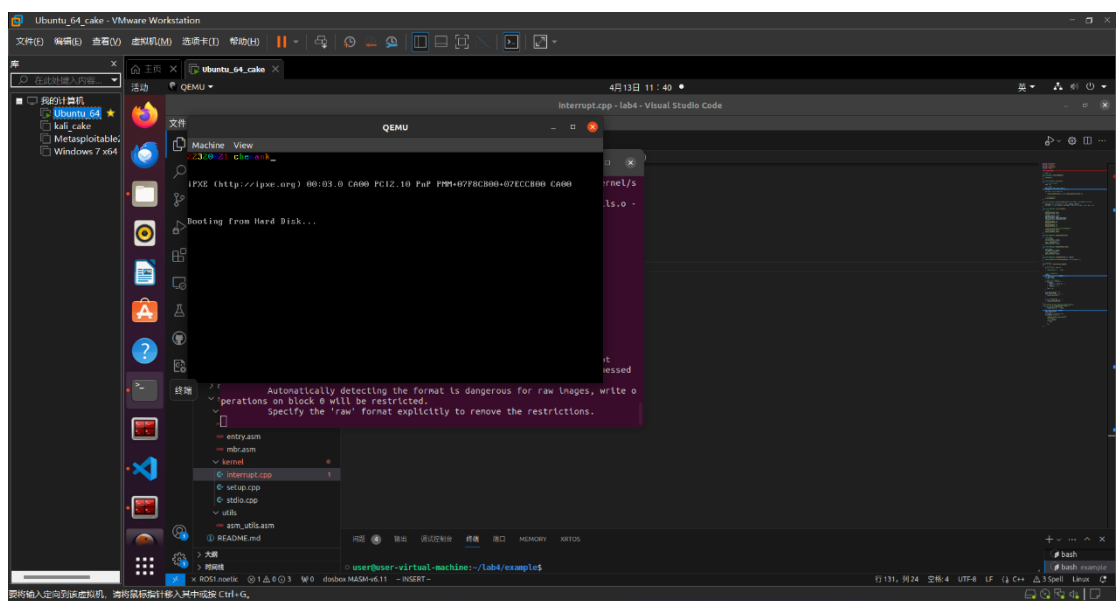
上图为复现 example 的结果截图。



上图任务 2 的实验结果截图。



上图为初始化 IDT 的结果截图。



上图为任务:4 实验截图。

5. 总结

通过本次实验我学习了混合编程以及操作系统中断的相关知识并进行了实践，受益匪浅。